

Pointwise Representation Certificates for Computation and Learning

Yunbei Xu
National University of Singapore
yunbei@nus.edu.sg

Preliminary note

Abstract

This note proposes a user-friendly and still-developing language for pointwise representation certificates. It does not identify runtime, sample complexity, generalization error, and regret. Instead, it separates three objects: the resource needed to discover or verify a representation, the local size of that representation, and the residual certificate obtained after the representation is available. The residual certificate has a type: it may be computational, statistical, information-theoretic, or optimization-theoretic. Average-case and smoothed analysis put distributions on inputs; parameterized complexity fixes explicit structural parameters. Pointwise representation certificates ask a different question: for this input, trained model, optimization trajectory, or interaction, has an admissible representation been discovered that is sufficient for a smaller residual problem? The note gives self-contained definitions and explains how backdoors, tree decompositions, local nonconvex certificates, pointwise neural-network generalization, data-driven algorithm design, learning-to-optimize, statistical-query restrictions, and Bellman log-potential search fit this typed certificate view without confusing computational complexity with statistical or information complexity.

1 The separation of quantities

A task is a family

$$\mathcal{P} = \{(\mathcal{X}_n, \mathcal{Y}_n, R_n) : n \geq 1\}.$$

Here $x \in \mathcal{X}_n$ is an input or instance, $y \in \mathcal{Y}_n(x)$ is an output or certificate, and $R_n(x, y) \in \{0, 1\}$ is the verification relation. For a language L , one may take $\mathcal{Y}_n = \{0, 1\}$ and $R_n(x, y) = \mathbf{1}\{y = \mathbf{1}_L(x)\}$. For an NP search problem, $R_n(x, w) = 1$ means that w is a valid witness.

The first convention is that the word *cost* is avoided unless its unit has been specified. The relevant quantity may be time, memory, verifier calls, communication, samples, feedback, regret, excess risk, proof length, or an optimization residual. These quantities should not be added unless a theorem supplies a common scale. A pointwise certificate is therefore typed.

Definition 1.1 (Typed performance criterion). *A typed performance criterion is a map $\mathbf{P}$ that assigns to an algorithm, estimator, policy, or solver a quantity with a specified unit. Examples include*

$$\text{Time}(A, x), \quad \text{Risk}(\hat{f}; \mathcal{D}), \quad \text{Reg}_T(\pi, \omega), \quad F_x(z) - \inf_z F_x(z),$$

where the first is computational, the second statistical, the third sequential/information-theoretic, and the fourth optimization-theoretic. A theorem must state which typed criterion is being bounded.

Thus representation complexity for computation and representation complexity for learning are not the same numerical object. They share a syntax—discover a representation, charge its size, solve a residual problem—but the residual theorem is different. In computational complexity the residual may be a running time or verifier-call bound. In learning it may be an excess-risk or generalization bound. In bandits and reinforcement learning it may be a regret or information bound. The common representation language is useful precisely because it keeps these distinctions explicit.

Classical worst-case complexity studies bounds of the form

$$\inf_A \sup_{x \in \mathcal{X}_n} \text{Time}(A, x)$$

([Roughgarden, 2021](#)). Average-case complexity fixes a distribution $\mathcal{D}_n$ on $\mathcal{X}_n$ and studies

$$\mathbb{E}_{X \sim \mathcal{D}_n} \text{Time}(A, X)$$

([Levin, 1986](#); [Yao, 1977](#)). This is analogous to using an external prior over inputs, but it is not a pointwise explanation of why a particular input is easy. Parameterized complexity fixes a structural parameter $\kappa(x)$ and proves

$$\text{Time}(A, x) \leq f(\kappa(x)) \text{ poly}(n)$$

([Downey and Fellows, 2013](#); [Cygan et al., 2015](#)). This is analogous to imposing a geometric or structural restriction on the instance. Pointwise representation certificates are more flexible: the useful representation may be discovered by the algorithm or interaction, but the discovery and verification resources must be charged ([Xu, 2026a](#)).

2 Admissible sufficient representations

Definition 2.1 (Admissible representation scheme). *For input length n , an admissible representation scheme $\mathcal{R}$ consists of:*

- (i) *a representation space $\mathcal{S}_n$;*
- (ii) *a discovery or construction procedure $G_{\mathcal{R},n}$ that maps an input or interaction history to a state $S \in \mathcal{S}_n$ and possibly a certificate ζ ;*
- (iii) *a residual interface, such as a verifier, feasible-operation map, oracle model, transition rule, objective primitive, statistical model, or Bellman update;*
- (iv) *a residual solver or residual theorem that uses only (S, ζ) and the residual interface.*

The scheme is admissible only if $\mathcal{R}$, $G_{\mathcal{R},n}$, and the residual solver or theorem are specified independently of the hidden answer or witness. If an oracle supplies solution information, that oracle must belong to the original problem and its resource usage must be charged.

Admissibility is essential. Without it, one could set S equal to the correct answer, a valid witness, or an optimal search path, and every problem would have zero residual difficulty. An analogy is PAC-Bayes prior blindness: the prior should not depend on, or already encode, the hypothesis being certified ([McAllester, 1999](#)).

Definition 2.2 (Sufficiency for a residual criterion). *Let $(S, \zeta) = G_{\mathcal{R},n}(x)$.*

(a) For a decision problem, (S, ζ) is sufficient if there is a computable map $D_{\mathcal{R}}$ such that

$$\mathbf{1}_L(x) = D_{\mathcal{R}}(S, \zeta).$$

(b) For an NP search problem, it is sufficient if the exact residual witness set is determined by the state:

$$\{w : R_n(x, w) = 1\} = W_{S, \zeta}.$$

(c) For an optimization problem, it is sufficient at accuracy ε if the residual certificate proves that the specified residual solver reaches an ε -optimal point, or proves the claimed local optimum condition, using only (S, ζ) .

(d) For a statistical learning problem, it is sufficient if the theorem's risk or excess-risk bound depends on the data, algorithm, and hypothesis only through (S, ζ) and stated residual quantities such as empirical error, approximation error, and optimization error.

(e) For an interactive decision problem with history H_{t-1} and state $S_t = \phi_t(H_{t-1})$, it is dynamically sufficient if the feasible operations, response law or adversarial response rule, loss or typed cost, and update close on the state:

$$\begin{aligned} \mathcal{A}_t(H_{t-1}) &= \mathcal{A}_t(S_t), \\ P_x(\cdot \mid H_{t-1}, a) &= P_{S_t}(\cdot \mid a), \quad c_x(H_{t-1}, a) = c(S_t, a), \\ S_{t+1} &= \tau_t(S_t, a_t, O_t). \end{aligned}$$

The full input or full interaction history is always sufficient. A compressed state is useful only after exact closure or a certified approximation has been verified.

Definition 2.3 (Typed pointwise representation certificate). For an admissible sufficient representation scheme $\mathcal{R}$, define the typed certificate

$$\text{Cert}_{\mathcal{R}}(x) := (\text{Disc}_{\mathcal{R}}(x), \text{Size}_{\mathcal{R}}(x), \text{Res}_{\mathcal{R}}(x)).$$

Here $\text{Disc}_{\mathcal{R}}(x)$ records the resource or evidence required to discover, compute, learn, or verify the representation; $\text{Size}_{\mathcal{R}}(x)$ records a description length, local entropy, rank, dimension, or other representation-size charge; and $\text{Res}_{\mathcal{R}}(x)$ records the residual certificate in its own type. These entries need not share units. A theorem that turns a certificate into a scalar bound must specify an evaluation functional

$$\Psi : (\text{Disc}, \text{Size}, \text{Res}) \mapsto \text{the typed quantity being bounded.}$$

Without such a Ψ , the tuple $\text{Cert}_{\mathcal{R}}(x)$ is a structured certificate rather than a single complexity number.

A pointwise certificate is therefore not always a scalar. The components of $(\text{Disc}, \text{Size}, \text{Res})$ may be numbers, but they may also be structured objects: an effective subspace, a local basin, a decomposition, a verifier, a Bellman potential, or a residual optimization certificate. The role of the theorem is to say which features of these objects enter the target quantity.

For example, in a running-time theorem the evaluation functional Ψ may convert discovery time, representation size, and residual dynamic-programming time into one computational resource bound. In a statistical risk theorem, Size may enter through a statistical penalty, while Disc records how the representation was found or certified and Res records empirical-risk, approximation, or optimization residuals. In pointwise deep-learning bounds, the learned effective eigenspace is part of the representation discovered by the algorithm, the associated local effective dimension contributes to Size, and the remaining empirical-risk, approximation, or optimization residual belongs to Res.

Admissible representations and quantifier order. This viewpoint is related to parameterized complexity, but it is not the same object. In ordinary parameterized complexity, a parameter $\kappa(x)$, structural decomposition, kernel, or backdoor is specified in advance, and the theorem bounds the residual computation in terms of that specified object. Here a representation may be selected for an instance or produced along an algorithmic trajectory, but only under an admissibility rule: the representation system must be specified before the hidden solution is known, the representation must be discoverable or verifiable, and all description, discovery, proof, verification, and residual-solving costs must be charged. Otherwise the representation is merely uncharged advice or oracle information.

Thus there are three distinct readings. A *hindsight* certificate explains why a particular instance has a useful representation. An *algorithm-selected* certificate explains why a fixed admissible algorithm, using its data, randomness, optimization path, or interaction transcript, produces a useful representation. A *minimax* certificate requires one admissible algorithmic or representation scheme to work uniformly before the hard instance or environment is chosen. These readings have different quantifier orders and should not be conflated.

This is also the right way to relate the present language to pointwise statistical theories. Recent pointwise deep-learning generalization and Gaussian-field separation results certify the complexity of an estimator, learned representation, or local subgeometry selected by the data and algorithm (Li and Xu, 2026; Xu, 2026a). Definition 2.3 uses the same pointwise spirit, but the representation certificate is attached to an instance, transcript, or algorithmic trajectory, and its meaning depends on whether the theorem is hindsight, distributional, algorithm-selected, or minimax.

Sequential decision making illustrates the minimax case (Xu, 2026b). Its primitive benchmark is not a hindsight pointwise complexity but the minimax risk. A representation used in an upper bound is part of the admissible algorithm: it may be a state-update rule, belief process, or Bellman certificate fixed before the environment is known and then updated from the interaction. For a fixed algorithm and fixed environment, the realized transcript and representation are generated jointly by the algorithm, the environment, and the observation randomness. Thus local or pointwise complexity is a diagnostic inside the minimax certificate; it is not a replacement for the minimax objective.

If a representation is supplied by an external prover rather than discovered by the algorithm, the model becomes closer to a proof system or interactive proof: the verifier’s checks, the proof length, the interaction cost, and the residual solver must all be included in the certificate (Goldwasser et al., 1989; Shamir, 1992). If these costs are not charged, the representation changes the problem rather than explaining the tractability of the original one.

3 How pointwise certificates coexist with hardness

Pointwise certificates can explain why particular instances are easy, but they do not invalidate worst-case hardness. To avoid ambiguity, we state the claim for a polynomially checkable search relation

$$R \subseteq \{0, 1\}^* \times \{0, 1\}^*,$$

with polynomially bounded witnesses and polynomial-time verifier. Its associated decision language is

$$L_R := \{x : \exists w \text{ such that } R(x, w) = 1\}.$$

Thus P and RP below refer to the decision language L_R . The class RP is the standard randomized polynomial-time class with one-sided error: no-instances are rejected with probability one, and yes-instances are accepted with probability at least a fixed positive constant, which can be amplified.

Proposition 3.1 below is a sanity check: if admissible pointwise certificates were uniformly polynomial for every input of an NP -hard problem, then the corresponding worst-case language would already fall into the existing standard classes P or RP .

Proposition 3.1 (Pointwise tractability does not collapse worst-case hardness). *Suppose R admits an admissible representation system with the following uniform polynomial guarantee. For every input x of length n , a discovery procedure finds a representation $s = \phi(x)$ in time $\text{poly}(n)$, the representation certificate is checkable in time $\text{poly}(n)$, and a residual deterministic solver, given (x, s) , correctly decides whether a witness exists and outputs a valid witness whenever one exists, all in time $\text{poly}(n)$. Then $L_R \in P$, and the search relation R is solvable in deterministic polynomial time.*

If instead the residual solver is randomized with one-sided error, meaning that it never outputs an invalid witness or accepts a no-instance, and on every yes-instance it outputs a valid witness with probability at least $1/2$, with polynomial running time and standard amplification, then $L_R \in RP$.

Proof. On input x , run the discovery procedure to obtain $s = \phi(x)$, verify the representation certificate, and then run the residual solver. In the deterministic case, the assumed soundness, completeness, and uniform polynomial running-time bounds give a polynomial-time decision algorithm for L_R and a polynomial-time search algorithm for R .

In the randomized case, reject unless the solver outputs a witness w with $R(x, w) = 1$. Soundness gives zero acceptance probability on no-instances. On yes-instances, the solver outputs a valid witness with probability at least $1/2$, and repeated independent trials amplify this success probability in polynomial time. Hence $L_R \in RP$. $\square$

Consequently, if L_R is NP -hard under polynomial-time many-one reductions, then a uniformly polynomial deterministic certificate system of the kind in Proposition 3.1 would imply $P = NP$. A uniformly polynomial one-sided randomized certificate system would imply $NP \subseteq RP$. Thus pointwise representation certificates should not be read as worst-case algorithms for all instances unless such a collapse is intended. Their useful role is pointwise, distributional, smoothed, or trajectory-dependent: some inputs, neighborhoods, training distributions, or algorithmic paths may have small admissible certificates even though the worst-case supremum remains large.

4 Pointwise computation dimension and generic-case complexity

Generic-case complexity is the closest existing notion to a pointwise theory of computation. Fix an input set $\mathcal{X} = \bigcup_n \mathcal{X}_n$, a size function, and probability measures π_n on inputs of size n , usually the uniform measures on finite spheres or balls. A partial algorithm is *generically correct* if it never gives a wrong answer and halts with a correct answer on a set $G \subseteq \mathcal{X}$ satisfying

$$\pi_n(G \cap \mathcal{X}_n) \longrightarrow 1.$$

If the halting time on $G \cap \mathcal{X}_n$ is polynomial, the problem is generically polynomial-time. This density-one viewpoint was introduced in group-theoretic decision problems and has since been developed in computability theory and symbolic logic (Kapovich et al., 2003; Gilman et al., 2007; Jockusch and Schupp, 2012). It is already powerful: for large classes of finitely generated groups, the word, conjugacy, and membership problems have linear-time generic-case complexity (Kapovich et al., 2003); Whitehead’s automorphism problem in a fixed free group has strongly linear generic-case complexity (Kapovich et al., 2006); and, on the logic side, the halting problem is decidable on a set of asymptotic probability one for a particular machine model (Hamkins and Miasnikov, 2006), while

other work shows strong generic undecidability phenomena and a Rice-type obstruction (Myasnikov and Rybalov, 2008; Rybalov, 2007). Thus generic-case complexity is already proven to have concrete consequences for classical group-theoretic and logical decision problems.

The limitation is that generic-case complexity is a *global* density-one statement. It asks whether there is one very large set of easy inputs. A pointwise representation certificate asks a finer question: for this input x , what representation was discovered, how much mass or code length does its local cell have, and what residual certificate solves the remaining problem? This leads to a pointwise computation dimension.

Definition 4.1 (Pointwise computation mass and dimension). *Fix an admissible representation scheme $\mathcal{R}$ and a length level $\mathcal{X}_n$ with reference measure π_n . Suppose the discovery procedure assigns to $x \in \mathcal{X}_n$ a representation state $S = S_{\mathcal{R}}(x)$. The representation cell of x is*

$$C_{\mathcal{R},n}(x) := \{z \in \mathcal{X}_n : S_{\mathcal{R}}(z) = S_{\mathcal{R}}(x)\}.$$

The pointwise computation mass and code dimension are

$$m_{\mathcal{R},n}(x) := \pi_n(C_{\mathcal{R},n}(x)), \quad d_{\mathcal{R},n}(x) := \log \frac{1}{m_{\mathcal{R},n}(x)}.$$

When π_n is uniform on $\mathcal{X}_n$, the normalized dimension is

$$\delta_{\mathcal{R},n}(x) := \frac{d_{\mathcal{R},n}(x)}{\log |\mathcal{X}_n|} = 1 - \frac{\log |C_{\mathcal{R},n}(x)|}{\log |\mathcal{X}_n|}.$$

These quantities are meaningful only together with the typed certificate

$$(\text{Disc}_{\mathcal{R}}(x), \text{Size}_{\mathcal{R}}(x), \text{Res}_{\mathcal{R}}(x)).$$

A large cell gives a small dimension, but it is useful only if the representation is admissibly discoverable and sufficient for the residual solver or theorem.

Proposition 4.2 (Generic-case complexity as a density-one specialization). *Suppose a partial algorithm solves a problem in time $t(n)$ on a generic set G . If membership in the generic stratum is itself admissibly certified, or if the halting of the partial algorithm is used as the certificate, then generic-case complexity is the special case of Definition 4.1 with a single large easy cell. More precisely, the two-cell representation*

$$S_G(x) = \mathbf{1}\{x \in G\}$$

has

$$d_{G,n}(x) = \log \frac{1}{\pi_n(G \cap \mathcal{X}_n)} = o(1)$$

for inputs in the generic cell, and the residual time on that cell is $t(n)$. Thus generic polynomial time is a global density-one certificate; pointwise computation dimension refines it by allowing many input-dependent cells, each with its own discovery cost, mass, and residual certificate.

Proof. For inputs in the easy cell, the cell mass is exactly $\pi_n(G \cap \mathcal{X}_n)$, which tends to one. Hence the logarithmic code dimension tends to zero. The residual algorithm is the generic partial algorithm restricted to the cell. The only additional requirement in the pointwise formulation is admissibility: the certificate must not encode the answer and the discovery or halting evidence must be charged. This proves the reduction from generic-case complexity to the pointwise-cell formulation. $\square$

Why computation uses representation cells rather than metric balls. Metric balls are natural in Hausdorff dimension, Gaussian fields, and pointwise statistical geometry because the metric is part of the analytic problem. Computation and logic are substantially different, because they usually do not come with a canonical metric under which nearby instances share residual difficulty. Two formulas, programs, or group words that differ by one symbol may have completely different behavior; conversely, two syntactically distant instances may share the same decomposition, normal-form certificate, easy random-walk behavior, or proof-search state. The natural local neighborhoods in computation are therefore representation cells, cylinders, strata, decompositions, or proof states: sets of inputs on which the same residual solver, verifier, or dynamic program is valid. Definition 4.1 is a discrete, combinatorial analogue of pointwise statistical dimension (Li and Xu, 2026; Xu, 2026a): it measures the mass of the local computational cell selected by the representation, rather than the radius of a metric ball.

Does pointwise computation dimension define a new complexity class? Not by itself. If one fixes an input ensemble, an admissible atlas, and thresholds on discovery, cell dimension, and residual resources, then one can define derived classes such as “polynomial pointwise dimension with polynomial residual time.” Such classes are useful diagnostics, but they are not canonical in the way P , RP , or NP are, because they depend on the chosen atlas and ensemble. Uniformly polynomial certificates for all inputs collapse back to ordinary polynomial-time solvability, as in Proposition 3.1; density-one polynomial certificates recover generic-case classes such as GenP; and distributional statements recover average-case or smoothed statements after taking expectations or tail probabilities of the typed certificate. The pointwise dimension should therefore be read as a certificate language and diagnostic scale, not as a new universal complexity class.

5 Examples of typed residual certificates

5.1 Parameterized residual dynamic programs

A backdoor set B for SAT is a sufficient representation when assigning the variables in B reduces the formula to a tractable class. If $|B| = k$ and the backdoor is discovered at resource $D(x)$, brute force over the backdoor gives

$$\text{Time}(x) \leq D(x) + 2^k \text{poly}(n).$$

A tree decomposition of width w gives, for many graphical and constraint problems,

$$\text{Time}(x) \leq D(x) + n \exp(O(w)).$$

These are standard parameterized-algorithmic examples of admissible sufficient representations: the representation is useful because it is checkable, discoverable or supplied with its discovery resource charged, and sufficient for the residual dynamic program (Downey and Fellows, 2013; Cygan et al., 2015; Roughgarden, 2021).

5.2 Local nonconvex optimization certificates

For nonconvex optimization, let $F_x : \mathbb{R}^m \rightarrow \mathbb{R}$ be the objective. A sufficient local optimization state may contain a region U_S , an initialization $z_0(S)$, and constants (L_S, μ_S) such that the algorithm stays in U_S , F_x has L_S -Lipschitz gradient on U_S , and the Polyak–Łojasiewicz inequality holds on U_S :

$$\frac{1}{2} \|\nabla F_x(z)\|^2 \geq \mu_S (F_x(z) - F_x^*(U_S)), \quad z \in U_S.$$

Then gradient descent with step size $1/L_S$ reaches error ε in

$$O\left(\frac{L_S}{\mu_S} \log \frac{F_x(z_0) - F_x^*(U_S)}{\varepsilon}\right)$$

iterations. This bypasses global nonconvex hardness only pointwise: the certificate proves that this instance and initialization lie in a tractable invariant basin. It does not claim that all basins can be found or that the full landscape is easy.

5.3 Statistical pointwise representations and deep learning

In supervised learning the target criterion is usually population risk, not runtime. A pointwise statistical certificate should therefore be written as an oracle inequality, not as a computational time bound. A typical form is

$$\begin{aligned} \text{Risk}(\hat{f}; \mathcal{D}) - \text{Risk}(f^*; \mathcal{D}) &\leq \text{Approx}(S) + \text{AlgErr}(\hat{f}; S) \\ &\quad + \text{Stat}(\text{Size}(S), n, \delta), \end{aligned}$$

with probability at least $1 - \delta$. Here S is the learned or selected representation, $\text{Approx}(S)$ is the bias or approximation price of using that representation, $\text{AlgErr}(\hat{f}; S)$ is the empirical or optimization residual such as an ERM gap, and $\text{Stat}(\text{Size}(S), n, \delta)$ is the statistical penalty. The discovery resource used to obtain S , for example training time, is a separate computational quantity unless the theorem explicitly combines it with risk.

Recent pointwise generalization theory for deep neural networks fits this template. The effective eigenspace or spectrum of the learned feature representation is a candidate state S ; its local effective dimension is a representation-size term; and the remaining residual is the pointwise generalization error within the represented class, together with the corresponding approximation and optimization errors (Li and Xu, 2026). The final theorem is a statistical generalization certificate. It becomes a computational certificate only after one also bounds the resource needed to find or verify the effective eigenspace and to control the empirical or optimization residual. Thus the representation language is shared, but the typed conclusion is different: feature-spectrum complexity controls a risk bound, not the running time of training unless training resources are separately charged.

5.4 Bellman log-potential search and sequential decisions

For planning, proof search, program synthesis, reasoning, bandits, or reinforcement learning, let Y_x be the correct answer, branch, proof, plan, policy, or certificate. Suppose a sufficient search state S_t maintains an admissible distribution q_t over candidate indices. A log-potential certificate uses

$$V_t(S_t) + \gamma \log \frac{1}{q_t(Y_x)}.$$

If each operation satisfies the verified Bellman inequality

$$c_t(S_t, A_t) + \mathbb{E}V_{t+1}(S_{t+1}) - V_t(S_t) \leq \gamma \mathbb{E} \log \frac{q_{t+1}(Y_x)}{q_t(Y_x)} + \varepsilon_t,$$

then telescoping gives

$$\mathbb{E} \sum_{t=1}^T c_t \leq V_1(S_1) + \gamma \log \frac{1}{q_1(Y_x)} + \sum_{t=1}^T \varepsilon_t,$$

up to a nonnegative terminal potential. The residual criterion here may be a search resource, a regret bound, or an information bound, depending on the problem. This is the log-potential form behind PAC–Bayes, exponential weights, and Bellman-sufficient information complexity for sequential decisions (Xu, 2026b). It is valid only when the state update, distribution update, and Bellman inequality are part of the certificate. The typed conclusion is a search-resource, regret, or information bound according to the problem statement; the computational cost of solving the Bellman program is separate unless it is also charged.

6 Existing theories through the typed lens

6.1 Average-case and smoothed analysis

Average-case complexity studies $\mathbb{E}_{X \sim \mathcal{D}_n} \text{Time}(A, X)$. A pointwise representation theorem instead proves a bound on a typed certificate $\text{Cert}_{\mathcal{R}}(X)$ and then studies its distribution. Smoothed analysis studies $X = \bar{x} + \xi$, with adversarial center $\bar{x}$ and random perturbation ξ (Spielman and Teng, 2004). A pointwise representation statement has the form

$$\sup_{\bar{x}} \mathbb{P}_{\xi} \{ \text{Cert}_{\mathcal{R}}(\bar{x} + \xi) \notin \mathcal{G}_u \} \leq \delta(u),$$

where $\mathcal{G}_u$ is the event that the discovery resource, representation size, and residual resource are within the claimed range. Perturbation does not remove worst-case hardness; it makes bad certificates unlikely under the perturbation model.

6.2 Data-driven algorithm design

Data-driven algorithm design chooses an algorithmic parameter from training instances and proves that its performance generalizes to future instances (Balcan, 2020; Balcan et al., 2021; Gupta and Roughgarden, 2020; Haussler, 1992). If Θ is a finite parameter class and the losses take values in $[0, 1]$, then, given m training instances $Z_1, \dots, Z_m$, with probability at least $1 - \delta$,

$$\sup_{\theta \in \Theta} \left| \mathbb{E}[\text{Cost}(A_{\theta}, Z)] - \frac{1}{m} \sum_{i=1}^m \text{Cost}(A_{\theta}, Z_i) \right| \lesssim \sqrt{\frac{\log |\Theta| + \log(1/\delta)}{m}}.$$

Here $\text{Cost}(A_{\theta}, Z)$ denotes the statistical loss incurred by algorithm A_{θ} on instance Z . This is a statistical generalization theorem about selecting an algorithmic parameter. It does not by itself describe the pointwise representation used by the selected algorithm on a future instance. A pointwise question can be asked after selection: which sufficient state does $A_{\hat{\theta}}$ discover on the new instance, what resource is required, and what residual problem remains?

6.3 Learning-to-optimize

Learning-to-optimize trains an update rule or iterative policy for a family of optimization problems (Chen et al., 2022; Sucker and Ochs, 2025; Sambharya and Stellato, 2025). The learned object is a computational policy, not just a static algorithmic parameter. A typed certificate has two layers. First, a statistical theorem controls how the learned optimizer generalizes across tasks. Second, a pointwise residual certificate controls convergence on a new task, for example by a local PL basin, a descent certificate, or a Bellman log-potential inequality. Deployment time, number of iterations, and memory are computational resources; task-generalization is a statistical quantity. They can both be studied, but they should not be merged without an explicit theorem.

6.4 Statistical-query restrictions

Statistical-query algorithms access expectations through noisy query oracles (Kearns, 1998; Feldman, 2017). This is a formal admissibility restriction: the state may contain only quantities estimable by the SQ oracle to the required tolerance. A high SQ dimension can be read as a lower bound on the existence of small SQ-accessible sufficient representations, even when sample-theoretic complexity alone is small. This is one of the cleanest links between statistical and computational complexity, because the statistical query interface is also a computational restriction.

7 Conclusion

Pointwise representation certificates are not a single replacement for computational complexity or statistical learning theory. It is a typed certificate language. Average-case analysis weights inputs; parameterized complexity restricts inputs by explicit structural parameters; pointwise representation certificates ask what useful state can be discovered on this input, trained model, or interaction, charge that discovery, and prove that the residual problem closes on the state. The framework separates computation, representation size, and statistical information. Computation discovers or exploits a representation; the representation has local size; and the residual certificate may be computational, statistical, optimization-theoretic, or information-theoretic. This provides a user-friendly and still-developing language for explaining the tractability of particular hard-looking instances, nonconvex objectives, combinatorial structures, learned optimizer states, neural representations, and reasoning searches, while remaining consistent with worst-case hardness.

References

- Maria-Florina Balcan. Data-driven algorithm design. In *Beyond the Worst-Case Analysis of Algorithms*, Cambridge University Press, 2020. See also arXiv:2011.07177.
- Maria-Florina Balcan, Dan DeBlasio, Travis Dick, Carl Kingsford, Tuomas Sandholm, and Ellen Vitercik. How much data is sufficient to learn high-performing algorithms? Generalization guarantees for data-driven algorithm design. In *STOC*, 2021.
- Tianlong Chen, Xiaohan Chen, Wuyang Chen, Howard Heaton, Jialin Liu, Zhangyang Wang, and Wotao Yin. Learning to optimize: A primer and a benchmark. *Journal of Machine Learning Research*, 23(189):1–59, 2022.
- Marek Cygan, Fedor V. Fomin, Łukasz Kowalik, Daniel Lokshantov, Dániel Marx, Marcin Pilipczuk, Michał Pilipczuk, and Saket Saurabh. *Parameterized Algorithms*. Springer, 2015.
- Rodney G. Downey and Michael R. Fellows. *Fundamentals of Parameterized Complexity*. Springer, 2013.
- Vitaly Feldman. A general characterization of the statistical query complexity. In *COLT*, 2017.
- Rishi Gupta and Tim Roughgarden. Data-driven algorithm design. *Communications of the ACM*, 63(6):87–94, 2020.
- Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

- David Haussler. Decision theoretic generalizations of the PAC model for neural net and other learning applications. *Information and Computation*, 100(1):78–150, 1992.
- Robert Gilman, Alexei G. Miasnikov, Alexei D. Myasnikov, and Alexander Ushakov. Report on generic case complexity. *Herald of Omsk University, Special Issue*, 103–110, 2007.
- Joel David Hamkins and Alexei Miasnikov. The halting problem is decidable on a set of asymptotic probability one. *Notre Dame Journal of Formal Logic*, 47(4):515–524, 2006.
- Carl G. Jockusch Jr. and Paul E. Schupp. Generic computability, Turing degrees, and asymptotic density. *Journal of the London Mathematical Society*, 85(2):472–490, 2012.
- Ilya Kapovich, Alexei Myasnikov, Paul Schupp, and Vladimir Shpilrain. Generic-case complexity, decision problems in group theory, and random walks. *Journal of Algebra*, 264(2):665–694, 2003.
- Ilya Kapovich, Paul Schupp, and Vladimir Shpilrain. Generic properties of Whitehead’s algorithm and isomorphism rigidity of random one-relator groups. *Pacific Journal of Mathematics*, 223(1):113–140, 2006.
- Michael Kearns. Efficient noise-tolerant learning from statistical queries. *Journal of the ACM*, 45(6):983–1006, 1998.
- Leonid A. Levin. Average case complete problems. *SIAM Journal on Computing*, 15(1):285–286, 1986.
- Shaojie Li and Yunbei Xu. Pointwise generalization in deep neural networks. arXiv:2605.18598, 2026.
- David A. McAllester. PAC-Bayesian model averaging. In *COLT*, 1999.
- Alexei G. Myasnikov and Alexander N. Rybalov. Generic complexity of undecidable problems. *Journal of Symbolic Logic*, 73(2):656–673, 2008.
- Alexander N. Rybalov. On the strongly generic undecidability of the halting problem. *Theoretical Computer Science*, 377(1–3):268–270, 2007.
- Tim Roughgarden, editor. *Beyond the Worst-Case Analysis of Algorithms*. Cambridge University Press, 2021.
- Adi Shamir. $IP = PSPACE$. *Journal of the ACM*, 39(4):869–877, 1992.
- Daniel A. Spielman and Shang-Hua Teng. Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. *Journal of the ACM*, 51(3):385–463, 2004.
- Rajiv Sambharya and Bartolomeo Stellato. Data-driven performance guarantees for classical and learned optimizers. *Journal of Machine Learning Research*, 26(171):1–49, 2025.
- Michael Sucker and Peter Ochs. Learning-to-optimize with PAC-Bayesian guarantees. *Journal of Machine Learning Research*, 26, 2025.
- Yunbei Xu. Pointwise complexity for Gaussian fields: Upper envelopes, algorithmic lower bounds, and separation. arXiv:2606.07931, 2026.
- Yunbei Xu. Bellman-sufficient information complexity. arXiv:2606.11171, 2026.
- Andrew Chi-Chih Yao. Probabilistic computations: Toward a unified measure of complexity. In *FOCS*, 1977.